diffmrc Tutorial

A hands-on walkthrough of the differentiable multireference stack

Version 0.1

This tutorial follows a user from installation to a complete state-averaged calculation with a benchmark comparison. It is paired with the *Manual*, which carries the full reference: the API, the methods and implementation chapters, the verification records, and the development architecture. The package is an early-stage research code; the tutorial states the limitations where they bite.

Contents

1 Introduction **2**
- 1.1 Who this tutorial is for . . . 2
- 1.2 What this tutorial covers . . . 2
- 1.3 Project status . . . 2

2 Installing and running **3**
- 2.1 Requirements . . . 3
- 2.2 Runtime settings that are not optional . . . 3
- 2.3 Checking the installation . . . 3

3 A first calculation **4**
- 3.1 The script . . . 4
- 3.2 What each piece does . . . 5
- 3.3 Reading the result . . . 5

4 State-averaged workflows **5**

5 Choosing an active space **5**

6 Convergence and diagnostics **5**

7 A benchmark walkthrough **5**

8 Limitations and how to check them **5**

1 Introduction

`diffmrc` is an end-to-end differentiable multireference electronic structure stack. It spans configuration interaction (CASCI), complete active space self-consistent field (CASSCF), state-averaged CASSCF, and second-order N-electron valence perturbation theory (NEVPT2). Every energy, nuclear gradient, and second derivative the package produces comes out of the same code path by automatic differentiation; there is no hand-derived derivative equation anywhere in the source.

The stack is built on PySCF and pyscfad, with JAX providing the differentiable array operations. All arithmetic is double precision. The package does not replace PySCF: the Gaussian-basis integrals, the reference SCF, and the benchmark comparisons all come from PySCF and pyscfad, while the differentiable multireference chain is original to this codebase.

1.1 Who this tutorial is for

You should be comfortable with the chemistry of active spaces and with running a Python quantum-chemistry calculation from a script. You do not need prior exposure to JAX: the tutorial introduces the two runtime settings that matter (`jax_enable_x64` and the CPU platform pin) right where they are first needed.

1.2 What this tutorial covers

- Chapter 2: installing the package and setting up a working environment.
- Chapter 3: a complete CASSCF nuclear-gradient calculation for water, explained line by line.
- Chapter **??**: state-averaged CASSCF with per-state NEVPT2, and how to read the root table it produces.
- Chapter **??**: choosing an active space, with the frontier-window rules the benchmark work forced us to write down.
- Chapter **??**: what the convergence diagnostics mean and how to act on them.
- Chapter **??**: a full benchmark walkthrough on a real target, end to end.
- Chapter **??**: where the method family and the present implementation stop being reliable.

The companion *Manual* carries what a tutorial should not: the API reference, protocol tables, the design of the differentiable chain and its second derivatives, the verification records, and the development architecture.

1.3 Project status

This is a research codebase, released to accompany a methods paper and to make its results reproducible. It is not a finished library: interfaces may change without notice, test coverage is partial, and the numerical behaviour of the orbital-optimization and perturbation layers depends on convergence settings that the code reports rather than enforces. Several limitations listed in Chapter **??** are properties of the method family and will not be removed by further development of this package. Use it for research, read the convergence diagnostics it returns, and do not treat it as a black box.

2 Installing and running

2.1 Requirements

Component	Version and role
Python	≥ 3.10
numpy, scipy	required; the analysis layer needs only these
pyscfad	$\geq 0.3.3$; provides the JAX-based `gto` and `scf`
pyscf	≥ 2.5; reference comparisons and integral drivers
jax	≥ 0.4; the differentiable array layer
matplotlib	only for the figure sources

`pyscfad` pins a specific JAX version, so install the package into a dedicated environment rather than into a shared one.

```
conda create -n diffmrc python=3.11
conda activate diffmrc
pip install -e ".[core,dev]"     # kernel, tests and lint tooling
```

For work on the analysis layer alone, `pip install -e ".[dev]"` is sufficient: that layer is plain NumPy and imports nothing from the numerical kernel.

2.2 Runtime settings that are not optional

Two settings must be established before the first JAX import.

```
import os
os.environ['JAX_PLATFORMS'] = 'cpu'     # before importing jax
for v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS',
          'OPENBLAS_NUM_THREADS', 'NUMEXPR_NUM_THREADS',
          'VECLIB_MAXIMUM_THREADS'):
    os.environ[v] = '8'

import jax
jax.config.update('jax_enable_x64', True)
```

`jax_enable_x64` must be set: the package is validated in double precision, and single-precision runs will produce silently wrong gradients. The CPU platform pin is deliberate. The stack has been developed and verified on CPU only; on a machine with a GPU the default JAX platform would select CUDA, where this build of pyscfad raises a solver error that has no bearing on the chemistry. Pin the platform for now; GPU support is not claimed.

The thread variables keep the BLAS layer inside the thread budget the driver assumes. They are not a performance suggestion: without them a single calculation can oversubscribe a shared node.

2.3 Checking the installation

```
pytest tests/ -v        # unit and integration tests
lint-imports            # verifies the layer boundaries
```

The layer contracts are worth a word even at install time. The package is organized in six layers (kernel, analysis, policy, driver, report, command line), and the import direction between them is enforced as a test. If `lint-imports` is unavailable in your environment, `tests/test_layer_contracts.py` performs the same check. A run that passes the suite but fails the contracts is a run the developers would want to see.

Two environment notes that cost us time, recorded so that they need not cost yours:

- pyscfad's molecule object cannot be traced inside `jax.jit`; the driver layer constructs plain-array callables for anything that is jitted. If you write your own jitted wrapper around a factory from `driver.nevpt2`, use the `theta`-only factories, which are built for that context.
- The first call into a heavy node of the chain compiles an XLA program and can take minutes; subsequent calls are fast. A slow first evaluation is expected, not a hang.

3 A first calculation

This chapter computes a CASSCF nuclear gradient for water at the 6-31G level, with an active space of two electrons in two orbitals (the highest occupied and lowest unoccupied orbital of the reference). It is the smallest complete run through the kernel: a reference SCF, an orbital-rotation optimization, and one nuclear gradient from automatic differentiation.

3.1 The script

```
import os
os.environ['JAX_PLATFORMS'] = 'cpu'
for v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS',
          'OPENBLAS_NUM_THREADS', 'NUMEXPR_NUM_THREADS',
          'VECLIB_MAXIMUM_THREADS'):
    os.environ[v] = '8'

import numpy as np
import jax
jax.config.update('jax_enable_x64', True)
from jax import numpy as jnp
from pyscfad import gto as pgto, scf as pscf

from diffmrc.core.rotation import block_rotation
from diffmrc.core.orblib import orbital_pairs
from diffmrc.core.optimizer import optimize_theta
from diffmrc.core.grad import (make_energy_fn, make_grad_fn,
                               solve_theta_star, nuclear_gradient)

mol = pgto.Mole()
mol.atom = 'O 0 0 0; H 0.7557 0 0.5880; H -0.7557 0 0.5880'
mol.basis = '6-31g'
mol.build()

nmo = mol.nao_nr()
ncore, nact, nele = 4, 2, 2
pairs = orbital_pairs(ncore, nact, nmo, include_cv=True)

E_cas = make_energy_fn(ncore, nact, nele)
mf = pscf.RHF(mol)
mf.kernel()
C_rhf = jnp.asarray(np.asarray(mf.mo_coeff))

h1e = mol.intor('int1e_kin') + mol.intor('int1e_nuc')
eri = mol.intor('int2e', aosym='s1')

E_theta = lambda t: E_cas(C_rhf @ block_rotation(t, nmo, pairs), h1e, eri,
                          mol.energy_nuc())
th_star, gnorm = solve_theta_star(mol, C_rhf, E_theta, len(pairs),
                                  optimize_theta, steps=2000,
                                  g_conv=1e-12)
```

```
g = nuclear_gradient(mol, make_grad_fn(nmo, pairs, th_star, E_cas, pscf))
print(g)          # shape (natm, 3), in Eh/bohr
```

3.2 What each piece does

The reference SCF is an ordinary spin-restricted Hartree–Fock from pyscfad, whose MO coefficients are the starting point of the chain. The rotation parameter vector `t` is turned into an orbital rotation by `block_rotation`; the pair list from `orbital_pairs` includes the core–virtual rotations, which are required for a correct stationary point — omitting them leaves the optimizer stopped at a point that is not stationary. `E_cas` is the CASCI energy of the rotated orbitals.

`solve_theta_star` runs the two-stage orbital optimization and returns the parameter vector *and* the gradient norm at the point it stopped at. Do not discard `gnorm`: it is the evidence of convergence, and the package reports stationarity from this quantity, not from an optimizer status code.

`nuclear_gradient` applies automatic differentiation to the whole converged chain with respect to the nuclear coordinates, with the orbital parameters held at their converged values. The mathematics of that construction is in the Manual (Chapter on the differentiable chain); for this tutorial the point is that no derivative expression was written anywhere in the package or in this script.

3.3 Reading the result

For this geometry the gradient is small but not zero — the atoms are close to, not at, the CASSCF/6-31G equilibrium. Two checks belong to every such run:

- `gnorm`, the orbital-rotation gradient norm, should sit at or below the threshold that `solve_theta_star` was asked for. If the refinement stalls above the threshold, the returned point carries the stall in its stopping reason; treat a stalled point as a candidate, not as a result.
- The energy is invariant under rotations within a degenerate orbital pair. If a molecule has exact degeneracies, small differences between two runs with different rotation orders are a property of the chart, not a failure of the code.

The next chapter moves from the ground state to several states at once — the setting in which the state-averaged machinery and the per-state perturbation theory come in.

4 State-averaged workflows

5 Choosing an active space

6 Convergence and diagnostics

7 A benchmark walkthrough

8 Limitations and how to check them